

```

- objects git: (main) git cat-file -p
ac6a9797c263b7582ef2afb6dca81a4934f5c89c
tree a3191707737bcf9357156ed70f879965e7795f69
author Anchal Agarwal 1705342062 +0530
committer Anchal Agarwal 1705342062 +0530

```

```

Adding first commit
- objects git: (main)

```

The code given above shows the content of a commit. It can be seen that it only contains SHA-1 of the current root working tree. To find any file or directory, it will start searching from root tree only.

You may wonder that since each commit contains a reference for the whole repository, does it create a new blob or tree object for each file and directory present in the repository?

To answer this question, I added a new file as *file2.txt* in the repository. The commit message I gave is 'Adding second commit'. The current Git history looks like what's given below:

```

commit bfbb79d72518aebdd5e7df7425bdcefc16e62552 (HEAD -> main)
Author: Anchal Agarwal
Date: Wed Jan 17 08:13:03 2024 +0530
    Adding second commit
commit ac6a9797c263b7582ef2afb6dca81a4934f5c89c
Author: Anchal Agarwal
Date: Mon Jan 15 23:37:42 2024 +0530
    Adding first commit

```

```

+ git_tutorial_dir git:(main) cd .git/objects
+ objects git:(main) git cat-file -p bfbb79d72518aebdd5e7df7425bdcefc16e62552
tree 99a55b8474dfc7939665c33dfe225054f3a25d66
parent ac6a9797c263b7582ef2afb6dca81a4934f5c89c
author Anchal Agarwal 1705459383 +0530
committer Anchal Agarwal 1705459383 +0530

Adding second commit
+ objects git:(main) git cat-file -p 99a55b8474dfc7939665c33dfe225054f3a25d66
100644 blob ec7ff284664263109178b5ea42eadfef0fa8e8e2 file1.txt
100644 blob 689c30ad13d9e86d6d8ba009d9fae2e3d9fd7cf3b file2.txt
+ objects git:(main)

```

Figure 4: New commit

Figure 4 shows the content of the new commit and its root tree object ID. If we print the content of the tree, we can see a new blob object has been created for *file2.txt*.

Let's create one more commit. This time I have edited *file1.txt* and committed the changes with the message 'Adding third commit'. The new commit ID is 543c5a825fc3ccad295b1c88f2a4ab6ce00c8791. The head is also changed and now points to the new commit ID.

The tree created for the new commit will look like what's shown in Figure 5.

Do observe the difference between both commit trees — only the blob object ID of *file1.txt* has changed and *file2.txt* refers to the same blob object. This is because nothing has been changed in *file2.txt* since the last commit. Therefore, we can conclude that a new blob or tree object is only created when the respective file or directory content changes. This strategy helps Git to store all the data.

```

→ objects git:(main) git cat-file -p bfbb79d72518aebdd5e7df7425bdcefc16e62552
tree 99a55b8474dfc7939665c33dfe225054f3a25d66
parent ac6a9797c263b7582ef2afb6dca81a4934f5c89c
author Anchal Agarwal 1705459383 +0530
committer Anchal Agarwal 1705459383 +0530

Adding second commit
→ objects git:(main) git cat-file -p 99a55b8474dfc7939665c33dfe225054f3a25d66
100644 blob ec7ff284664263109178b5ea42eadfef0fa8e8e2 file1.txt
100644 blob 689c30ad13d9e86d6d8ba009d9fae2e3d9fd7cf3b file2.txt
→ objects git:(main)
→ objects git:(main)
→ objects git:(main) git cat-file -p 543c5a825fc3ccad295b1c88f2a4ab6ce00c8791
tree 230739027f584fdb03b602cdbc897d73aa794eb2
parent bfbb79d72518aebdd5e7df7425bdcefc16e62552
author Anchal Agarwal 1705461923 +0530
committer Anchal Agarwal 1705461923 +0530

Adding third commit
→ objects git:(main) git cat-file -p 230739027f584fdb03b602cdbc897d73aa794eb2
100644 blob 5fe815768a481d47b0a42f97ee8626adcde1f8de file1.txt
100644 blob 689c30ad13d9e86d6d8ba009d9fae2e3d9fd7cf3b file2.txt
→ objects git:(main)

```

Previous commit

Different Blob ID for file1.txt

HEAD commit

Figure 5: Commit tree